

🌟 Logical Operators in C

□ Definition:

Logical operators in C are used to **combine two or more conditions (expressions)** and make decisions based on **true (1)** or **false (0)** results.

They are mostly used in **if**, **while**, and **for** statements to check **multiple conditions at once**.

💬 How It Works

- **AND (&&)** → All conditions must be true.
Example:
if (age > 18 && age < 60) → Only true if both are true.
Think: "Both must agree."
- **OR (||)** → Only one condition needs to be true.
Example:
if (marks > 90 || marks == 100) → True if any one condition is true.
Think: "At least one must agree."
- **NOT (!)** → Reverses the condition.
Example:
if (!(a == b)) → True if a is not equal to b.
Think: "Opposite day!"

Logical Operators		
Operator	Description	Example
&&	AND	x=6 y=3 x<10 && y>1 Return True
	OR	x=6 y=3 x==5 y==5 Return False
!	NOT	x=6 y=3 !(x==y) Return True

💻 Example Program

```
#include <stdio.h>

int main() {
    int a = 5, b = 10, c = 15;

    printf("a < b && b < c : %d\n", (a < b && b < c));
    printf("a > b || b < c : %d\n", (a > b || b < c));
    printf("!(a == b)      : %d\n", !(a == b));

    return 0;
}
```

📄 Output:

```
CSS

a < b && b < c : 1
a > b || b < c : 1
!(a == b)      : 1
```

★ Assignment Operators in C

Definition:

Assignment operators in C are used to assign values to variables.

The most common one is `=`, but there are also shortcut forms that combine arithmetic and assignment together.

```
x = 5;    // assigns 5 to x
x += 3;   // adds 3 to x (same as x = x + 3)
```

Here the **value of the right-side expression** is first **calculated**, and the **result** is then **stored into the left-side variable** using the assignment operator (`=`).

⚙️ List of Assignment Operators

Operator	Example	Meaning / Description	Equivalent To	Example (if x = 10)	Result
<code>=</code>	<code>x = y</code>	Assigns value of <code>y</code> to <code>x</code>	—	<code>x = 5</code>	<code>x</code> becomes 5
<code>+=</code>	<code>x += y</code>	Adds <code>y</code> to <code>x</code>	<code>x = x + y</code>	<code>x += 3</code>	13
<code>-=</code>	<code>x -= y</code>	Subtracts <code>y</code> from <code>x</code>	<code>x = x - y</code>	<code>x -= 2</code>	8
<code>*=</code>	<code>x *= y</code>	Multiplies <code>x</code> by <code>y</code>	<code>x = x * y</code>	<code>x *= 2</code>	20
<code>/=</code>	<code>x /= y</code>	Divides <code>x</code> by <code>y</code>	<code>x = x / y</code>	<code>x /= 5</code>	2
<code>%=</code>	<code>x %= y</code>	Stores remainder of <code>x / y</code>	<code>x = x % y</code>	<code>x %= 3</code>	1

💻 Example Program

```
#include <stdio.h>

int main() {
    int x = 10, y = 5;

    printf("Initial value of x = %d\n", x);

    x += y;    // x = x + y
    printf("After x += y, x = %d\n", x);

    x -= y;    // x = x - y
    printf("After x -= y, x = %d\n", x);

    x *= y;    // x = x * y
    printf("After x *= y, x = %d\n", x);

    x /= y;    // x = x / y
    printf("After x /= y, x = %d\n", x);

    return 0;
}
```

OUTPUT

```
Initial value of x = 10
After x += y, x = 15
After x -= y, x = 10
After x *= y, x = 50
After x /= y, x = 10
```

Step-by-Step Explanation

1. `x = 10` → assigns 10 to `x`.
2. `x += y` → adds 5 to `x` → now `x` = 15.
3. `x -= y` → subtracts 5 → now `x` = 10.
4. `*x = y` → multiplies `x` by 5 → now `x` = 50.
5. `x /= y` → divides `x` by 5 → now `x` = 10.

💡 Fun Tip

`x += 10;` ➡ is the same as ➡ `x = x + 10;`

Increment and Decrement Operators in C

◆ Definition

Increment and decrement operators are unary operators (work on a single variable). They are used to increase or decrease the value of a variable by 1.

◆ Types				
Operator	Name	Description	Example	Result
<code>++</code>	Increment Operator	Increases the value of a variable by 1	<code>++x</code> or <code>x++</code>	Adds 1 to x
<code>--</code>	Decrement Operator	Decreases the value of a variable by 1	<code>--x</code> or <code>x--</code>	Subtracts 1 from x

◆ Forms of Increment / Decrement			
Form	Description	Example	Execution
Pre-increment (<code>++x</code>)	Increments first, then uses the value	<code>y = ++x;</code>	x increases first, then assigned to y
Post-increment (<code>x++</code>)	Uses the value first, then increments	<code>y = x++;</code>	x assigned to y, then increases
Pre-decrement (<code>--x</code>)	Decreases first, then uses the value	<code>y = --x;</code>	x decreases first, then assigned to y
Post-decrement (<code>x--</code>)	Uses the value first, then decreases	<code>y = x--;</code>	x assigned to y, then decreases

Note

- `++` and `--` can only be applied to **variables**, not constants or expressions.

✗ `++(x + y);` → Invalid

✓ `++x;` → Valid

EXAMPLE PROGRAM

```
#include <stdio.h>

int main() {
    int x = 5, y;

    printf("Initial value of x: %d\n", x);

    y = ++x; // Pre-increment: x becomes 6 first, then y = 6
    printf("After pre-increment (++x): x = %d, y = %d\n", x, y);

    y = x++; // Post-increment: y = 6 first, then x becomes 7
    printf("After post-increment (x++): x = %d, y = %d\n", x, y);

    y = --x; // Pre-decrement: x becomes 6 first, then y = 6
    printf("After pre-decrement (--x): x = %d, y = %d\n", x, y);

    y = x--; // Post-decrement: y = 6 first, then x becomes 5
    printf("After post-decrement (x--): x = %d, y = %d\n", x, y);

    return 0;
}
```

OUTPUT

```
Initial value of x: 5
After pre-increment (++x): x = 6, y = 6
After post-increment (x++): x = 7, y = 6
After pre-decrement (--x): x = 6, y = 6
After post-decrement (x--): x = 5, y = 6
```